

WAL logs analysis

Analysis of write-ahead log of a PostgreSQL database cluster can be performed using the `pg_waldump` tool, however there are considerations on how to safely use `pg_waldump` with production data.

Security Compliance guideline regarding WAL analysis:

- **IMPORTANT: you should NEVER download WAL files into your personal workstation**
- WAL data contain users transactions, hence might contain **red data**
- Production **red data** should only be handled only within controled environments that follow Gitlab's Security Compliance
- Users should be granted least privilege access to Gitlab.com production (`gprd gcp project`) environment.
- If a user doesn't have access to `gprd` it might request download of WAL files for debug/troubleshooting purposes into a node in the `db-benchmarking` environment.

How to Fetch WALs from a Production environment into a working VM in the db-benchmarking environment

This is the procedure to fetch WALs from a GPRD database into a postgres VM in the db-benchmarking environment

1. Have wal-g installed on the VM by assigning the `gitlab_walg::default` recipe to the proper Chef role

- Create the VM or run `chef-client` on the node to apply the recipe

2. Once wal-g is installed, manually configure the GPRD the gcs settings and credentials on it

- As root create a `/etc/wal-g.d/env-gprd` directory with the same content of `/etc/wal-g.d/env`

```
mkdir /etc/wal-g.d/env-gprd/  
cp /etc/wal-g.d/env/* /etc/wal-g.d/env-gprd/
```
- Edit `/etc/wal-g.d/env-gprd/WALG_GS_PREFIX` with the content from the GPRD environment (copy from a GPRD server)
- Edit `/etc/wal-g.d/env-gprd/GOOGLE_APPLICATION_CREDENTIALS` to point to `/etc/wal-g.d/gcs-gprd.json`
- Define the GPRD GCS credentials in the `/etc/wal-g.d/gcs-gprd.json` file (copy from a GPRD server)
- Change ownership of the new GPRD env and credential files (**IMPORTANT, DON'T SKIP THIS STEP**)

```
chown gitlab-psql /etc/wal-g.d/env-gprd/*
chmod 600 /etc/wal-g.d/env-gprd/*
chown gitlab-psql /etc/wal-g.d/gcs-gprd.json
chmod 600 /etc/wal-g.d/gcs-gprd.json
```

- Create the `/var/opt/gitlab/wal_restore` directory

```
mkdir /var/opt/gitlab/wal_restore
chown gitlab-psql.gitlab-psql /var/opt/gitlab/wal_restore
chmod 770 /var/opt/gitlab/wal_restore
```

3. Download/install the following script in the VM

- Download the `fetch_last_wals_from_gcs_into_dir.sh` script
- Edit any variables in the script according with the environment you want to fetch WALs from
- Run the `fetch_last_wals_from_gcs_into_dir.sh` script as `gitlab-psql` user in a TMUX session

```
tmux
sudo su - gitlab-psql
cd /var/opt/gitlab/wal_restore
/usr/local/bin/fetch_last_wals_from_gcs_into_dir.sh
```

The script can take several hours to execute, because the list of WAL files with `gsutil ls -l` is a very-very-slow process.

3.1 If `fetch_last_wals_from_gcs_into_dir.sh` fails during wal-g execution

Most common issues are:

- issues with the GCS authentication file, check the credentials file `/etc/wal-g.d/gcs-gprd.json` content and permission
- issues with the GCS bucket file, check the bucket file `/etc/wal-g.d/env-gprd/WALG_GS_PREFIX` content and permission

After fixing, you don't need to list files again, as it's a very slow process. Check if the content of the `${RESTORE_DIR}/wal_list.download` file is populated and then just execute the 2nd part of the script to perform `wal-g wal-fetch`.

```
WALG_ENV_DIR="/etc/wal-g.d/env-gprd"
GCS_WAL_LOC="gs://gitlab-gprd-postgres-backup/pitr-walg-main-v14/wal_005"
RESTORE_DIR="/var/opt/gitlab/wal_restore"
WAL_LIST_DL_FILE="wal_list.download"
WAL_COUNT="150"
```

```
if [ $(wc -l < ${RESTORE_DIR}/${WAL_LIST_DL_FILE}) -gt 0 ]
then
  for WAL_FILE in $(cat ${RESTORE_DIR}/${WAL_LIST_DL_FILE})
  do
```

```
        /usr/bin/envdir $WALG_ENV_DIR /opt/wal-g/bin/wal-g wal-fetch $WAL_FILE $RESTORE_DIR/$WAL_FILE
    done
fi
```

4. Include the requestor user into the gitlab-psql group to grant access into the /var/opt/gitlab/wal_restore directory

```
usermod -a -G gitlab-psql <user>
```

5. Clean the gprd credential files and GCS prefix file

```
truncate -s 0 /etc/wal-g.d/gcs-gprd.json
truncate -s 0 /etc/wal-g.d/env-gprd/WALG_GS_PREFIX
```

Using pg_waldump

Check the documentation for pg_waldump

Example

```
/usr/lib/postgresql/14/bin/pg_waldump -p /var/opt/gitlab/wal_restore <startseg>
```

Where <startseg> is the segment name of the archived WAL, which is the file name of a file in /var/opt/gitlab/wal_restore